

---

## VPN\_Tunneling

---

## LAB Setup Environment

Run the SEED VM>> open Firefox in that>> go to the seed labs and then click on the seed labs >> network security > > Select the VPN Tunneling Lab >>> Download the .zip file.

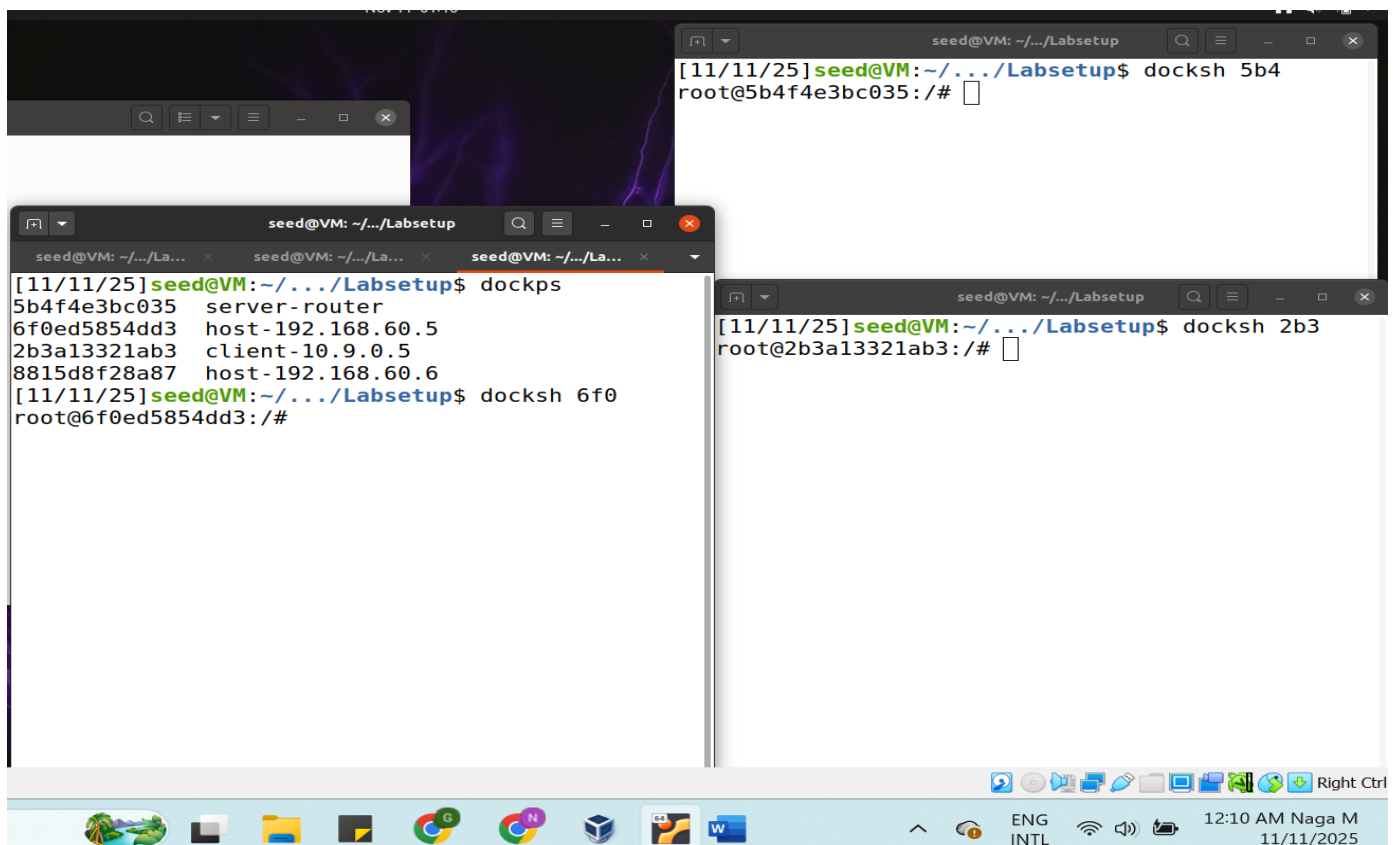
Copy this zip to Documents (any other folder in VM), here I moved it to documents >> VPN >> there unzip the labsetup.zip file. From here right click and open the Terminal then we can directly move to Terminal.

Now, We need to Set up Containers, and then use the "dock ps" command to find out the ID of the container



```
seed@VM: ~/.../Labsetup
[11/11/25]seed@VM:~/.../Labsetup$ dockps
5b4f4e3bc035  server-router
6f0ed5854dd3  host-192.168.60.5
2b3a13321ab3  client-10.9.0.5
8815d8f28a87  host-192.168.60.6
[11/11/25]seed@VM:~/.../Labsetup$
```

### Task 1: Network Setup



```
seed@VM: ~/.../Labsetup
[11/11/25]seed@VM:~/.../Labsetup$ docksh 5b4
root@5b4f4e3bc035:/#

seed@VM: ~/.../La...
[11/11/25]seed@VM:~/.../Labsetup$ dockps
5b4f4e3bc035  server-router
6f0ed5854dd3  host-192.168.60.5
2b3a13321ab3  client-10.9.0.5
8815d8f28a87  host-192.168.60.6
[11/11/25]seed@VM:~/.../Labsetup$ docksh 6f0
root@6f0ed5854dd3:/#

seed@VM: ~/.../Labsetup
[11/11/25]seed@VM:~/.../Labsetup$ docksh 2b3
root@2b3a13321ab3:/#
```

## Verify Network Configuration

```
seed@VM: ~/.../Labsetup
5b4f4e3bc035 server-router
6f0ed5854dd3 host-192.168.60.5
2b3a13321ab3 client-10.9.0.5
8815d8f28a87 host-192.168.60.6
[11/11/25]seed@VM:~/.../Labsetup$ docksh 6f0
root@6f0ed5854dd3:/# ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
8: eth0@if9: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:c0:a8:3c:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.5/24 brd 192.168.60.255 scope global eth0
        valid_lft forever preferred_lft forever
```

From the above we can see that interface IP as 192.168.60.5

```
seed@VM: ~/.../Labsetup
len 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
10: eth0@if11: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
oup default
    link/ether 02:42:c0:a8:3c:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 10.9.0.11/24 brd 10.9.0.255 scope global eth0
        valid_lft forever preferred_lft forever
14: eth1@if15: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
oup default
    link/ether 02:42:c0:a8:3c:0b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.11/24 brd 192.168.60.255 scope global eth1
        valid_lft forever preferred_lft forever
root@5b4f4e3bc035:/#
```

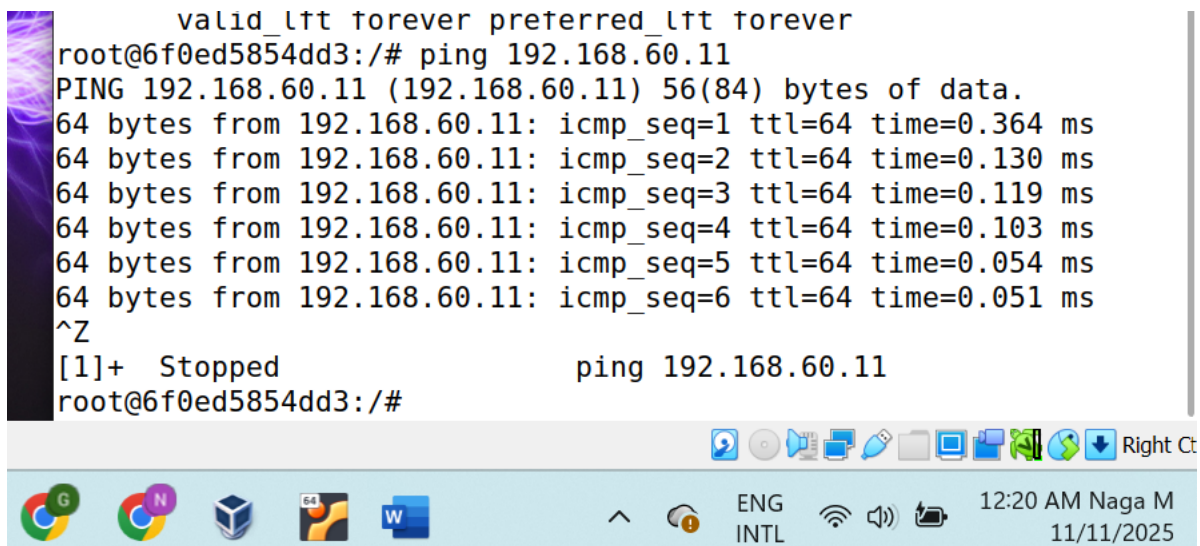
We should see TWO interfaces:

- One with IP 192.168.60.11 (facing Internet)
- One with IP 10.9.0.11 (facing private network)

## Perform Connectivity Tests:

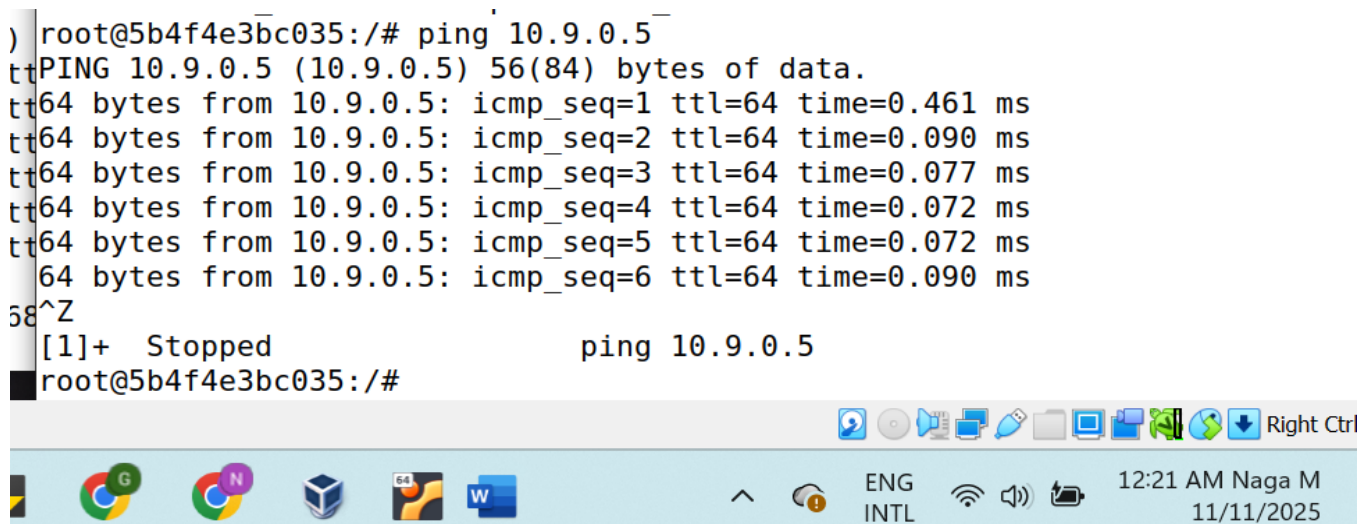
**Test 1: Host U → VPN Server (WORKED)** : Successful ping replies. This confirms basic Internet connectivity.

```
valid_lft forever preferred_lft forever
root@6f0ed5854dd3:/# ping 192.168.60.11
PING 192.168.60.11 (192.168.60.11) 56(84) bytes of data.
64 bytes from 192.168.60.11: icmp_seq=1 ttl=64 time=0.364 ms
64 bytes from 192.168.60.11: icmp_seq=2 ttl=64 time=0.130 ms
64 bytes from 192.168.60.11: icmp_seq=3 ttl=64 time=0.119 ms
64 bytes from 192.168.60.11: icmp_seq=4 ttl=64 time=0.103 ms
64 bytes from 192.168.60.11: icmp_seq=5 ttl=64 time=0.054 ms
64 bytes from 192.168.60.11: icmp_seq=6 ttl=64 time=0.051 ms
^Z
[1]+  Stopped                  ping 192.168.60.11
root@6f0ed5854dd3:/#
```



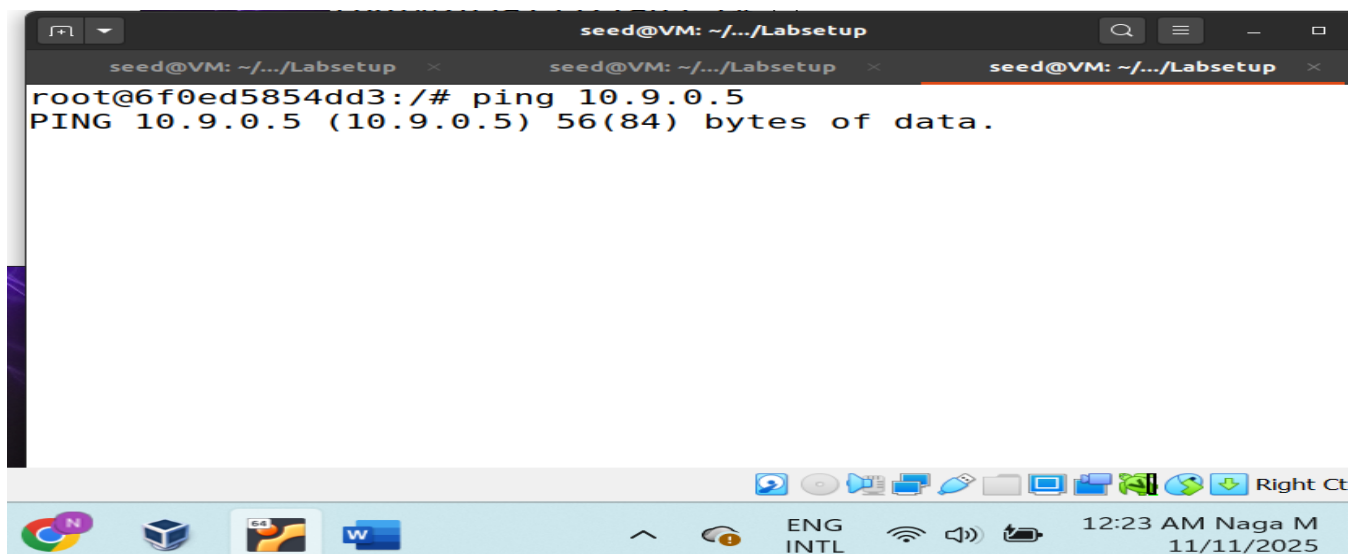
VPN Server → Host V ( WORKED ) : Successful ping replies. This confirms private network is working.

```
) root@5b4f4e3bc035:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=64 time=0.461 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=64 time=0.090 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=64 time=0.077 ms
64 bytes from 10.9.0.5: icmp_seq=4 ttl=64 time=0.072 ms
64 bytes from 10.9.0.5: icmp_seq=5 ttl=64 time=0.072 ms
64 bytes from 10.9.0.5: icmp_seq=6 ttl=64 time=0.090 ms
^Z
[1]+  Stopped                  ping 10.9.0.5
root@5b4f4e3bc035:/#
```



Test 3: Host U → Host V ( FAILED ) : because There's no direct route between these networks

```
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
seed@VM: ~/.../Labsetup
root@6f0ed5854dd3:/# ping 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
```

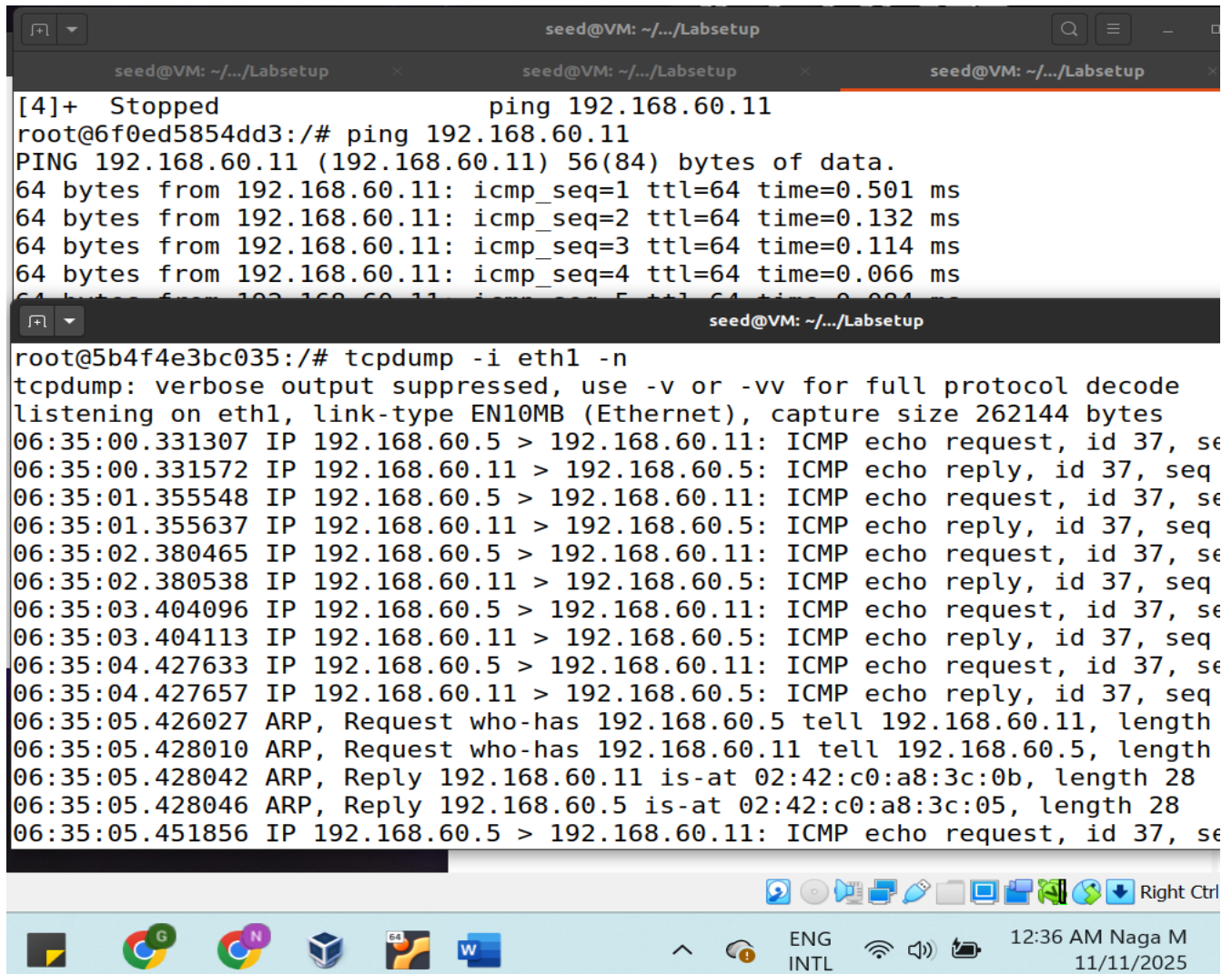




## Capturing Traffic on Both Networks

Step 1: Sniff on Internet Network (eth1)

On VPN Server, monitor traffic: `tcpdump -i eth1 -n`, and then generate traffic **from Host U** while `tcpdump` is running in server:



The screenshot shows a terminal window titled 'seed@VM: ~/.../Labsetup'. The user has stopped a ping command and then started a `tcpdump -i eth1 -n` command. The output shows several ICMP echo requests and replies between 192.168.60.5 and 192.168.60.11, as well as ARP requests and replies. The terminal window is part of a desktop environment with a taskbar at the bottom showing various application icons and system status information.

```
seed@VM: ~/.../Labsetup
[4]+  Stopped                  ping 192.168.60.11
root@6f0ed5854dd3:/# ping 192.168.60.11
PING 192.168.60.11 (192.168.60.11) 56(84) bytes of data.
64 bytes from 192.168.60.11: icmp_seq=1 ttl=64 time=0.501 ms
64 bytes from 192.168.60.11: icmp_seq=2 ttl=64 time=0.132 ms
64 bytes from 192.168.60.11: icmp_seq=3 ttl=64 time=0.114 ms
64 bytes from 192.168.60.11: icmp_seq=4 ttl=64 time=0.066 ms
64 bytes from 192.168.60.11: icmp_seq=5 ttl=64 time=0.084 ms

seed@VM: ~/.../Labsetup
root@5b4f4e3bc035:/# tcpdump -i eth1 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth1, link-type EN10MB (Ethernet), capture size 262144 bytes
06:35:00.331307 IP 192.168.60.5 > 192.168.60.11: ICMP echo request, id 37, seq 37, length 60
06:35:00.331572 IP 192.168.60.11 > 192.168.60.5: ICMP echo reply, id 37, seq 37, length 60
06:35:01.355548 IP 192.168.60.5 > 192.168.60.11: ICMP echo request, id 37, seq 38, length 60
06:35:01.355637 IP 192.168.60.11 > 192.168.60.5: ICMP echo reply, id 37, seq 38, length 60
06:35:02.380465 IP 192.168.60.5 > 192.168.60.11: ICMP echo request, id 37, seq 39, length 60
06:35:02.380538 IP 192.168.60.11 > 192.168.60.5: ICMP echo reply, id 37, seq 39, length 60
06:35:03.404096 IP 192.168.60.5 > 192.168.60.11: ICMP echo request, id 37, seq 40, length 60
06:35:03.404113 IP 192.168.60.11 > 192.168.60.5: ICMP echo reply, id 37, seq 40, length 60
06:35:04.427633 IP 192.168.60.5 > 192.168.60.11: ICMP echo request, id 37, seq 41, length 60
06:35:04.427657 IP 192.168.60.11 > 192.168.60.5: ICMP echo reply, id 37, seq 41, length 60
06:35:05.426027 ARP, Request who-has 192.168.60.5 tell 192.168.60.11, length 28
06:35:05.428010 ARP, Request who-has 192.168.60.11 tell 192.168.60.5, length 28
06:35:05.428042 ARP, Reply 192.168.60.11 is-at 02:42:c0:a8:3c:0b, length 28
06:35:05.428046 ARP, Reply 192.168.60.5 is-at 02:42:c0:a8:3c:05, length 28
06:35:05.451856 IP 192.168.60.5 > 192.168.60.11: ICMP echo request, id 37, seq 42, length 60
```

Step 2: Sniff on Private Network (eth0)

On VPN Server, monitor traffic: `tcpdump -i eth0 -n`, and then generate traffic **from VPN server(open another terminal)** while `tcpdump` is running in server:

```
seed@VM: ~/.../Labsetup
[11/11/25]seed@VM:~/.../Labsetup$ docksh 5b4
root@5b4f4e3bc035:/# ping -c 3 10.9.0.5
PING 10.9.0.5 (10.9.0.5) 56(84) bytes of data.
64 bytes from 10.9.0.5: icmp_seq=1 ttl=64 time=0.196 ms
64 bytes from 10.9.0.5: icmp_seq=2 ttl=64 time=1.35 ms
64 bytes from 10.9.0.5: icmp_seq=3 ttl=64 time=0.125 ms

--- 10.9.0.5 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2023ms
rtt min/avg/max/mdev = 0.125/0.558/1.354/0.563 ms
root@5b4f4e3bc035:/#

06:35:09.548443 IP 192.168.60.11 > 192.168.60.5: ICMP echo reply, id 37, seq
^Z
[4]+  Stopped                  tcpdump -i eth1 -n
root@5b4f4e3bc035:/# tcpdump -i eth0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
06:41:43.244373 IP6 fe80::42:dcff:fed1:d09a > ff02::2: ICMP6, router sollicita
06:42:56.816139 IP 10.9.0.11 > 10.9.0.5: ICMP echo request, id 29, seq 1, len
06:42:56.816305 IP 10.9.0.5 > 10.9.0.11: ICMP echo reply, id 29, seq 1, lengt
06:42:57.835855 IP 10.9.0.11 > 10.9.0.5: ICMP echo request, id 29, seq 2, ler
06:42:57.837178 IP 10.9.0.5 > 10.9.0.11: ICMP echo reply, id 29, seq 2, lengt
06:42:58.838698 IP 10.9.0.11 > 10.9.0.5: ICMP echo request, id 29, seq 3, ler
06:42:58.838794 IP 10.9.0.5 > 10.9.0.11: ICMP echo reply, id 29, seq 3, lengt
06:43:01.836283 ARP, Request who-has 10.9.0.5 tell 10.9.0.11, length 28
06:43:01.836473 ARP, Request who-has 10.9.0.11 tell 10.9.0.5, length 28
06:43:01.836492 ARP, Reply 10.9.0.11 is-at 02:42:0a:09:00:0b, length 28
06:43:01.836504 ARP, Reply 10.9.0.5 is-at 02:42:0a:09:00:05, length 28
```

After seeing these two outputs, we can Conclude that Packet sniffing works on both network interfaces.

## Task 2: Create and Configure TUN Interface

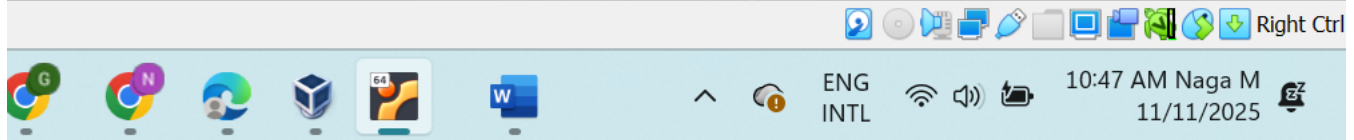
The objective is to create and configure a TUN virtual network interface on Host U (host-192.168.60.5). This TUN interface will be the endpoint of our VPN tunnel.

After running the existing tun.py file , we can see that the Interface name is tun0

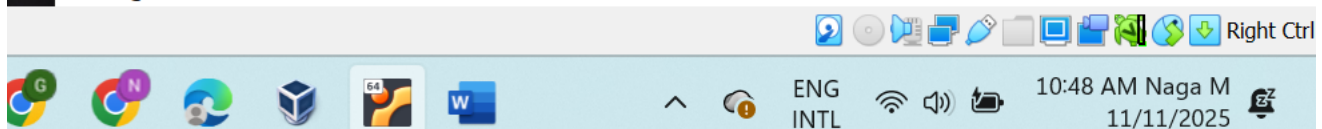
```
time.sleep(10)
root@c55b9914f1ac:/volumes# chmod a+x tun.py
root@c55b9914f1ac:/volumes# tun.py
Interface Name: tun0
..
```

Task 2.a: Name of the Interface: so instead of using tun as the prefix of the interface name, use your last name as the prefix.

```
root@c55b9914f1ac:/volumes# chmod a+x tun.py
root@c55b9914f1ac:/volumes# tun.py
Interface Name: Mahankali0
^Z
[3]+  Stopped                  tun.py
root@c55b9914f1ac:/volumes#
```



```
[11/11/25]seed@VM:~/.../Labsetup$ docksh c55
root@c55b9914f1ac:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN grc
ult qlen 500
    link/none
3: Mahankali0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DC
p default qlen 500
    link/none
46: eth0@if47: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue s
group default
    link/ether 02:42:c0:a8:3c:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.5/24 brd 192.168.60.255 scope global eth0
        valid_lft forever preferred_lft forever
root@c55b9914f1ac:/#
```



Task 2.b: Set up the TUN Interface: First, we need to assign an IP address to it. Second, we need to bring up the interface, because the interface is still in the down state. We can use the following two commands for the configuration:

```
[11/11/25]seed@VM:~/.../Labsetup$ docksh c55
root@c55b9914f1ac:/# ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group
t qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
2: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN grc
ult qlen 500
    link/none
3: Mahankali0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DC
p default qlen 500
    link/none
46: eth0@if47: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue s
group default
    link/ether 02:42:c0:a8:3c:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.5/24 brd 192.168.60.255 scope global eth0
        valid_lft forever preferred_lft forever
root@c55b9914f1ac:/#
```

we should see your TUN interface (like tun0 or mahankali0) but it will be in **DOWN** state with **no IP address**. Run these two commands in the **VPN Client container**:

*# Configure the TUN interface*

```
os.system("ip addr add 192.168.53.99/24 dev {}".format(ifname))
```

```
os.system("ip link set dev {} up".format(ifname))
```

```
root@c55b9914f1ac:/volumes# chmod a+x tun.py
root@c55b9914f1ac:/volumes# tun.py
TUN interface tun1 configured and brought up
```



```

2: tun0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DOWN gr
ult qlen 500
    link/none
3: Mahankali0: <POINTOPOINT,MULTICAST,NOARP> mtu 1500 qdisc noop state DI
p default qlen 500
    link/none
4: Mahankali1: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc
state UNKNOWN group default qlen 500
    link/none
    inet 192.168.53.99/24 scope global Mahankali1
        valid_lft forever preferred_lft forever
5: tun1: <POINTOPOINT,MULTICAST,NOARP,UP,LOWER_UP> mtu 1500 qdisc fq_cod
UNKNOWN group default qlen 500
    link/none
    inet 192.168.53.99/24 scope global tun1
        valid_lft forever preferred_lft forever
46: eth0@if47: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue
group default
    link/ether 02:42:c0:a8:3c:05 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.60.5/24 brd 192.168.60.255 scope global eth0
        valid_lft forever preferred_lft forever
root@c55b9914f1ac:/#

```

Before those Commands, the tun interface was down there no IP address, but not it got UP and had initiated a ip address.

### Task 2.c: Read from the TUN Interface

1. On Host U, ping a host in the 192.168.53.0/24 network. What are printed out by the tun.py program? What has happened? Why?

```

while True:
    # Get a packet from the tun interface
    packet = os.read(tun, 2048)
    if packet:
        ip = IP(packet)
        print(ip.summary())

```

```

^G Get Help    ^O Write Out  ^W Where Is   ^K Cut Text   ^J Justify    ^C Cur Pos
^X Exit        ^R Read File  ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line

```

12:45 PM Naga M  
11/11/2025

```

root@e0d879b0f952:/volumes# ls
tun_task2.c.py
root@e0d879b0f952:/volumes# chmod a+x tun_task2.c.py
root@e0d879b0f952:/volumes# tun_task2.c.py
interface
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw

```

However, in this case, we can see that packets (ICMP pings) coming into the TUN interface. after ping 192.168.153.1

Question2. On Host U, ping a host in the internal network 192.168.60.0/24, Does tun.py print out anything? Why?

```

root@e0d879b0f952:/volumes# tun_task2.c.py
interface

```

```

root@e0d879b0f952:/# ping 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
64 bytes from 192.168.60.1: icmp_seq=1 ttl=64 time=0.270 ms
64 bytes from 192.168.60.1: icmp_seq=2 ttl=64 time=0.099 ms
64 bytes from 192.168.60.1: icmp_seq=3 ttl=64 time=0.093 ms
64 bytes from 192.168.60.1: icmp_seq=4 ttl=64 time=0.094 ms
64 bytes from 192.168.60.1: icmp_seq=5 ttl=64 time=0.076 ms
64 bytes from 192.168.60.1: icmp_seq=6 ttl=64 time=0.087 ms
64 bytes from 192.168.60.1: icmp_seq=7 ttl=64 time=0.096 ms
^Z

```

```

[2]+  Stopped                  ping 192.168.60.1
root@e0d879b0f952:/#

```



From the output we observed, Here, we pinged 192.168.60.1 (in the internal network), printed **NOTHING**. Because in the internal network communication we don't need VPN. So it doesn't use the tun.

## Task 2.d: Write to the TUN Interface

This is the step where we actually send the packets you read from the client's TUN interface to the server (VPN tunnel) and possibly receive packets back.

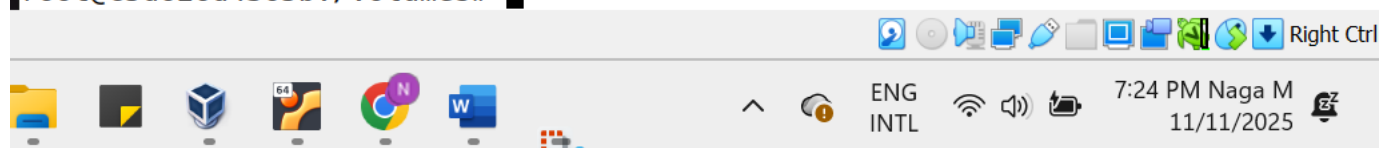
```
seed@VM: ~/.../Labsetup seed@VM: ~/.../Labsetup seed@VM: ~/.../Labsetup
root@d20dela46feb:/volumes# chmod a+x tun.py
root@d20dela46feb:/volumes# tun.py
interface Name:Naga0
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.1 echo-reply 0 / Raw
^Z
[2]+ Stopped tun.py
root@d20dela46feb:/volumes#
```

[11/11/25]seed@VM:~/.../Labsetup\$ docksh d20  
root@d20dela46feb:/# ping 192.168.53.1  
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.  
64 bytes from 192.168.53.1: icmp\_seq=1 ttl=64 time=3.35 ms  
64 bytes from 192.168.53.1: icmp\_seq=2 ttl=64 time=4.18 ms  
64 bytes from 192.168.53.1: icmp\_seq=3 ttl=64 time=3.78 ms  
64 bytes from 192.168.53.1: icmp\_seq=4 ttl=64 time=3.78 ms  
64 bytes from 192.168.53.1: icmp\_seq=5 ttl=64 time=3.08 ms  
64 bytes from 192.168.53.1: icmp\_seq=6 ttl=64 time=3.34 ms  
64 bytes from 192.168.53.1: icmp\_seq=7 ttl=64 time=2.72 ms  
64 bytes from 192.168.53.1: icmp\_seq=8 ttl=64 time=2.24 ms  
64 bytes from 192.168.53.1: icmp\_seq=9 ttl=64 time=3.20 ms  
^Z  
[1]+ Stopped ping 192.168.53.1  
root@d20dela46feb:/#

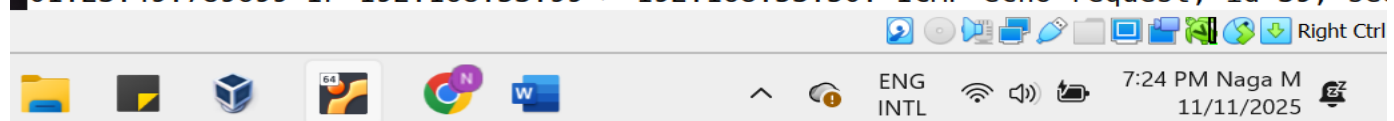
You should see successful ping replies! Even though there is no *real* machine at 192.168.53.99, your program is intercepting the request and fabricating a reply. The ping command believes it has communicated with a real host. This is the core proof that our code works as expected.

2. instead of writing an IP packet to the interface, write some arbitrary data to the interface, and report your observation.

```
root@c5d626d4365b:/volumes# chmod a+x tun.py
root@c5d626d4365b:/volumes# tun.py
interface Name:Naga0
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
IP / ICMP 192.168.53.99 > 192.168.53.50 echo-request 0 / Raw
^Z
[2]+  Stopped                  tun.py
root@c5d626d4365b:/volumes#
```



```
[11/11/25]seed@VM:~/.../Labsetup$ docksh c5d
root@c5d626d4365b:/# tcpdump -i Naga0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on Naga0, link-type RAW (Raw IP), capture size 262144 bytes
01:23:43.612265 IP 192.168.53.99 > 192.168.53.50: ICMP echo request, id 39, seq
1, length 64
01:23:43.613554 IP [|ip]
01:23:44.636356 IP 192.168.53.99 > 192.168.53.50: ICMP echo request, id 39, seq
2, length 64
01:23:44.642827 IP [|ip]
01:23:45.660126 IP 192.168.53.99 > 192.168.53.50: ICMP echo request, id 39, seq
3, length 64
01:23:45.661784 IP [|ip]
01:23:46.684137 IP 192.168.53.99 > 192.168.53.50: ICMP echo request, id 39, seq
4, length 64
01:23:46.685717 IP [|ip]
01:23:47.708099 IP 192.168.53.99 > 192.168.53.50: ICMP echo request, id 39, seq
5, length 64
01:23:47.709209 IP [|ip]
01:23:48.756381 IP 192.168.53.99 > 192.168.53.50: ICMP echo request, id 39, seq
6, length 64
01:23:48.761094 IP [|ip]
01:23:49.789699 IP 192.168.53.99 > 192.168.53.50: ICMP echo request, id 39, seq
```



When arbitrary data is written to the TUN interface instead of properly formatted IP packets, the system generates **malformed IP packets** that tcpdump cannot properly decode.

```
[11/11/25]seed@VM:~/.../Labsetup$ docksh c5d
root@c5d626d4365b:/# ping 192.168.53.50
PING 192.168.53.50 (192.168.53.50) 56(84) bytes of data.

^C
--- 192.168.53.50 ping statistics ---
89 packets transmitted, 0 received, 100% packet loss, time 90497ms

root@c5d626d4365b:/#
```

The experiment confirms that the TUN interface operates at the IP layer and requires valid IP packet formatting for proper network communication.

### **TASK3: Send the IP Packet to VPN Server Through a Tunnel**

```
root@d25bf83de0bf:/volumes# chmod a+x tun_client.py
root@d25bf83de0bf:/volumes# tun_client.py
Interface Name: Naga0
^Z
[2]+  Stopped                  tun_client.py
root@d25bf83de0bf:/volumes#
```

The below output was printed in VPN server: When we ping from Host U (e.g., ping 192.168.53.50), the packet goes through the tunnel.

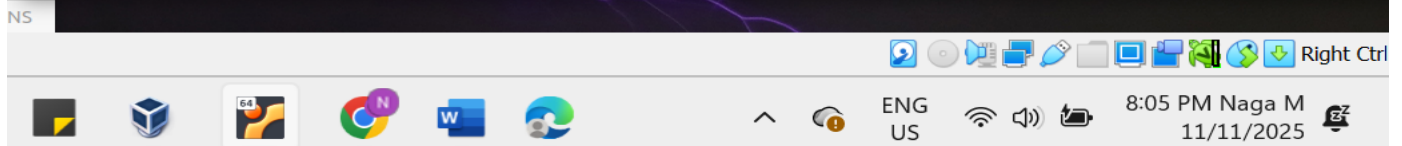
The **tun\_client.py** on Host U: Captures the ICMP ping packet from its TUN interface, Encapsulates it in UDP and sends to 10.9.0.11:9090 (VPN Server)

The **tun\_server.py**:

**First line:** 10.9.0.11:9090 --> 0.0.0.0:9090 : Shows the UDP packet arrived from client IP 10.9.0.11 port 9090 to server 0.0.0.0:9090. This is the outer UDP encapsulation

**Second line:** Inside: 192.168.53.99 --> 192.168.53.1 :Shows the **inner IP packet** that was encapsulated, Source: 192.168.53.99 (server's TUN interface IP), Destination: 192.168.53.50(the IP you pinged).

```
root@538247d37cd6:/volumes# tun_server.py
Interface Name: Naga0
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
10.9.0.5:49483 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.53.50
```



What we pinged in Host-U

```
[11/11/25]seed@VM:~/.../Labsetup$ docksh d2
root@d25bf83de0bf:/# ping 192.168.53.50
PING 192.168.53.50 (192.168.53.50) 56(84) bytes of data.
^C
--- 192.168.53.50 ping statistics ---
59 packets transmitted, 0 received, 100% packet loss, time 59389ms

root@d25bf83de0bf:/#
```



The tunnel is working - we're successfully encapsulating IP packets inside UDP and transporting them between client and server.

**Q. Access the hosts inside the private network 192.168.60.0/24 using the tunnel.**

**Lets ping 192.168.60.10**

```
root@f7c704cec9f7:/# ping 192.168.60.10
PING 192.168.60.10 (192.168.60.10) 56(84) bytes of data.
^Z
[1]+  Stopped                  ping 192.168.60.10
root@f7c704cec9f7:/# ping 192.168.60.1
PING 192.168.60.1 (192.168.60.1) 56(84) bytes of data.
^Z
[2]+  Stopped                  ping 192.168.60.1
root@f7c704cec9f7:/#
```

```
[11/11/25]seed@VM:~/.../Labsetup$ docksh d8
root@d8db18ee4ac5:/# cd volumes
root@d8db18ee4ac5:/volumes# ls
tun.py  tun_2d.py  tun_client.py  tun_server.py  tun_task2d.py
root@d8db18ee4ac5:/volumes# python3 tun_server.py
Interface Name: Naga0
```

```
[11/11/25]seed@VM:~/.../Labsetup$ docksh f7
root@f7c704cec9f7:/# cd volumes
root@f7c704cec9f7:/volumes# ls
tun.py  tun_2d.py  tun_client.py  tun_server.py  tun_task2d.py
root@f7c704cec9f7:/volumes# python3 tun_client.py
Interface Name: Naga0
```

Nothing is printed as the interface is not configured to pass the packets through its private network.

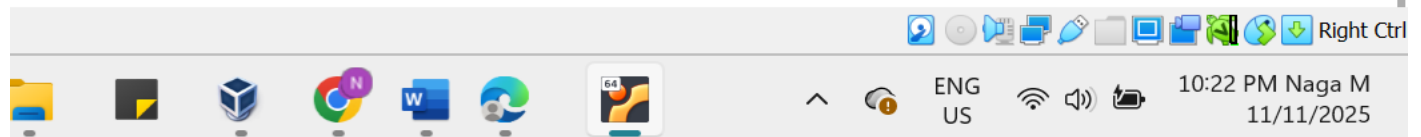
And now, After adding the routing commands, we ping the private network again to see the packets now passing through Naga0 interface as shown below:



```

root@4f88903b22b3:/# ip route add 192.168.60.10 dev Naga0
root@4f88903b22b3:/# ip route
default via 10.9.0.1 dev eth0
10.9.0.0/24 dev eth0 proto kernel scope link src 10.9.0.5
192.168.53.0/24 dev Naga0 proto kernel scope link src 192.168.53.99
192.168.60.0/24 dev Naga0 scope link
192.168.60.10 dev Naga0 scope link
root@4f88903b22b3:/# █

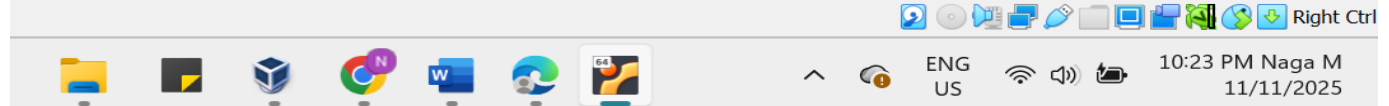
```



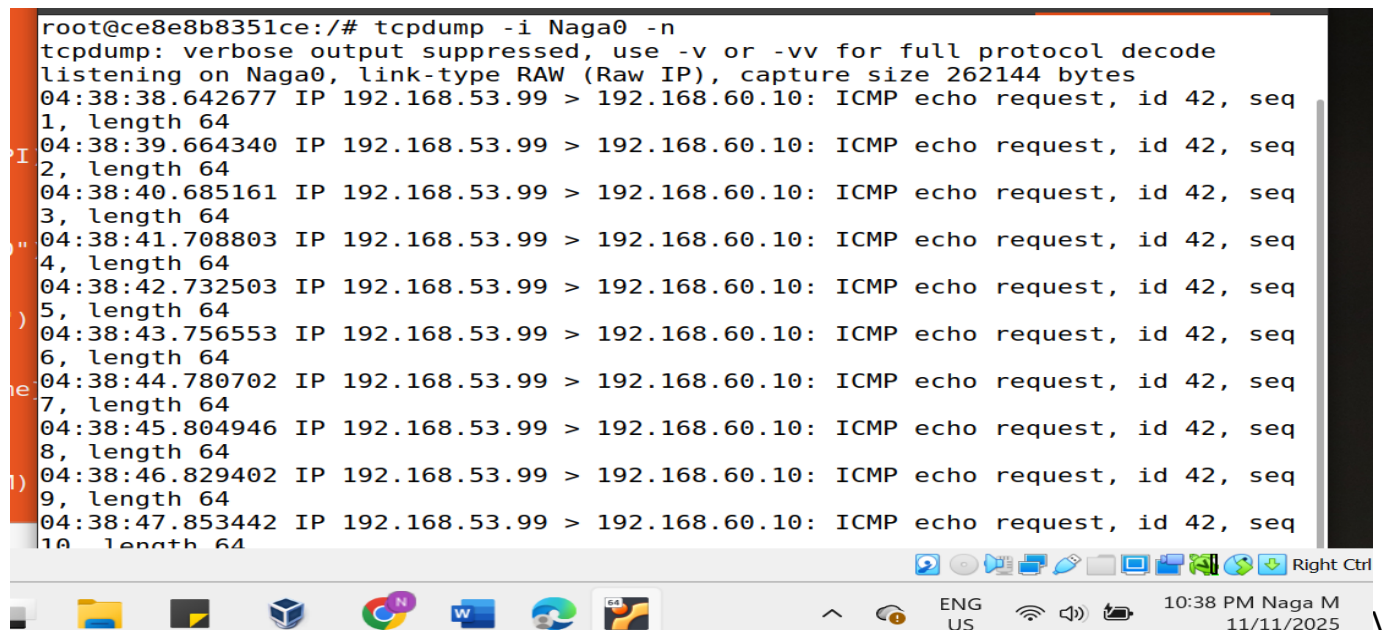
```

root@a4ae9a56f9c0:/volumes# python3 tun_server.py
Interface Name: Naga0
10.9.0.5:47558 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.60.10
10.9.0.5:47558 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.60.10
10.9.0.5:47558 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.60.10
10.9.0.5:47558 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.60.10
10.9.0.5:47558 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.60.10
10.9.0.5:47558 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.60.10
10.9.0.5:47558 --> 0.0.0.0:9090
  Inside: 192.168.53.99 --> 192.168.60.10

```



Now when you ping 192.168.60.10 from Host U, packets go into tun, get sent via UDP to VPN Server and should appear at tun\_server.py. we can confirm that it by using tcpdump.



```

root@ce8e8b8351ce:/# tcpdump -i Naga0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on Naga0, link-type RAW (Raw IP), capture size 262144 bytes
04:38:38.642677 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
1, length 64
04:38:39.664340 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
2, length 64
04:38:40.685161 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
3, length 64
04:38:41.708803 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
4, length 64
04:38:42.732503 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
5, length 64
04:38:43.756553 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
6, length 64
04:38:44.780702 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
7, length 64
04:38:45.804946 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
8, length 64
04:38:46.829402 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
9, length 64
04:38:47.853442 IP 192.168.53.99 > 192.168.60.10: ICMP echo request, id 42, seq
10, length 64

```

## Task 4 — Set up the VPN server-side TUN and forwarding

You can see in `docker-compose.yml` that the router container has the following entry:

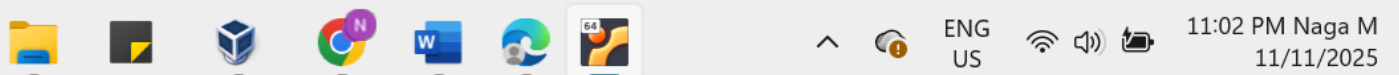
```
cap_add:
- ALL
devices:
- "/dev/net/tun:/dev/net/tun"
sysctls:
- net.ipv4.ip_forward=1
volumes:
- ./volumes:/volumes
networks:
  net-10.9.0.0:
    ipv4_address: 10.9.0.11
```

```
root@0985aefbc693: /# ./l3setup.py docker 09
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=5.30 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=4.48 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=9.94 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=6.57 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=63 time=2.15 ms
64 bytes from 192.168.60.5: icmp_seq=6 ttl=63 time=6.55 ms

--- 192.168.60.5 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5015ms
rtt min/avg/max/mdev = 2.152/5.829/9.940/2.366 ms
root@0985aefbc693: /#
```

After ping Host V from Host U. The ICMP echo request packets should eventually arrive at Host V through the tunnel. , here is the proof,

```
[1]+ Stopped nano tun_server.py  
root@708671e586e9:/volumes# python3 tun_server.py  
Interface Name: Naga0  
From socket <==: 192.168.53.99 --> 192.168.60.5  
From tun ==>: 192.168.60.5 --> 192.168.53.99  
From socket <==: 192.168.53.99 --> 192.168.60.5  
From tun ==>: 192.168.60.5 --> 192.168.53.99  
From socket <==: 192.168.53.99 --> 192.168.60.5  
From tun ==>: 192.168.60.5 --> 192.168.53.99  
From socket <==: 192.168.53.99 --> 192.168.60.5  
From tun ==>: 192.168.60.5 --> 192.168.53.99  
From socket <==: 192.168.53.99 --> 192.168.60.5  
From tun ==>: 192.168.60.5 --> 192.168.53.99  
From socket <==: 192.168.53.99 --> 192.168.60.5  
From tun ==>: 192.168.60.5 --> 192.168.53.99  
From socket <==: 192.168.53.99 --> 192.168.60.5  
From tun ==>: 192.168.60.5 --> 192.168.53.99  
From socket <==: 192.168.53.99 --> 192.168.60.5
```

The screenshot shows the bottom portion of a Windows operating system interface. The taskbar contains several application icons: File Explorer, a dark square icon, a cube icon, Google Chrome with a purple 'N' extension, Microsoft Word, a circular icon with a person silhouette, and a game icon labeled '64'. On the right side of the taskbar, there are system tray icons including network status, volume control, and a battery level indicator. The system clock displays the time as 11:02 PM and the date as 11/11/2025. The text 'Right Ctrl' is partially visible next to the battery icon.

```
root@0985aefbc693:/volumes# python3 tun_client.py
```

```
Interface Name: Naga0
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

```
From socket <==: 192.168.60.5 --> 192.168.53.99
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

```
From socket <==: 192.168.60.5 --> 192.168.53.99
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

```
From socket <==: 192.168.60.5 --> 192.168.53.99
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

```
From socket <==: 192.168.60.5 --> 192.168.53.99
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

```
From socket <==: 192.168.60.5 --> 192.168.53.99
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

```
From socket <==: 192.168.60.5 --> 192.168.53.99
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

```
From socket <==: 192.168.60.5 --> 192.168.53.99
```

```
From tun ==>: 192.168.53.99 --> 192.168.60.5
```

ENG US 11:04 PM Naga M 11/11/2025

```
[1]+ Stopped nano docker-compose.yml
```

```
[11/11/25]seed@VM:~/.../Labsetup$ docksh 09
```

```
root@0985aefbc693:/# tcpdump -i Naga0 -n
```

```
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
```

```
listening on Naga0, link-type RAW (Raw IP), capture size 262144 bytes
```

```
05:02:15.258741 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 38, seq 1, length 64
```

```
05:02:15.262860 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 38, seq 1, length 64
```

```
05:02:16.263676 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 38, seq 2, length 64
```

```
05:02:16.267366 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 38, seq 2, length 64
```

```
05:02:17.264450 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 38, seq 3, length 64
```

```
05:02:17.269144 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 38, seq 3, length 64
```

```
05:02:18.267348 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 38, seq 4, length 64
```

```
05:02:18.270893 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 38, seq 4, length 64
```

ENG US 11:05 PM Naga M 11/11/2025

**By using telnet:** here, we can see that the connection was successfully established.

```
root@0985aefbc693:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
bc606b206638 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)
```

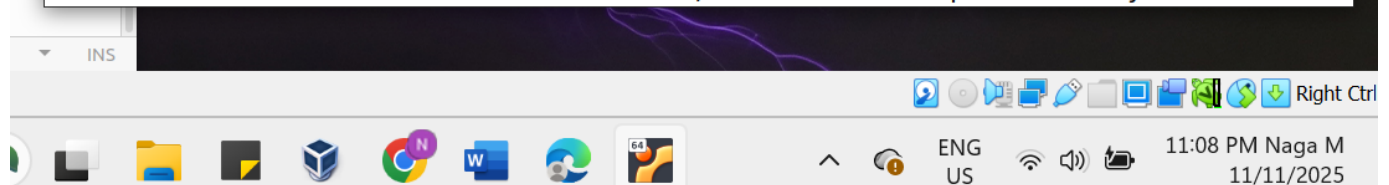
```
* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:        https://ubuntu.com/advantage
```

This system has been minimized by removing packages and content that are not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software; the exact distribution terms for each program are described in the individual files in /usr/share/doc/\*/copyright.

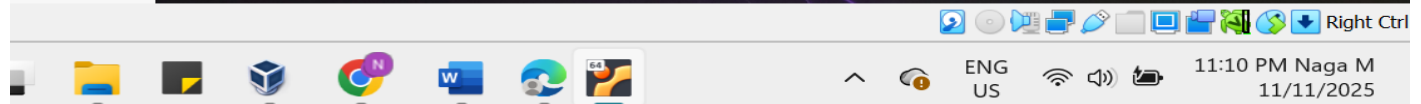
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by



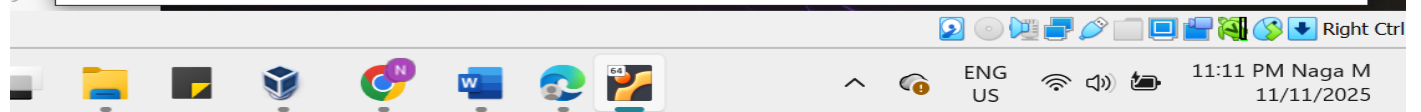


## Task5: Handling Traffic in Both Directions

```
seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x seed@VM: ~/... x
From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
From tun ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
From tun ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
From tun ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
From tun ==>: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
From tun ==>: 192.168.53.99 --> 192.168.60.5
```



```
root@708671e586e9:/volumes# python3 tun_server.py
Interface Name: Naga0
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.53.99
From socket <==: 192.168.53.99 --> 192.168.60.5
```





## Tcpdump output:

```
ack 25, win 509, options [nop,nop,TS val 3560914663 ecr 3782230669], length 15
[|telnet WILL SUPPRESS GO AHEAD, DO NAWs, DO LFLOW, DONT LINEMODE, WILL STATUS [|telnet]
->
by 05:07:48.290296 IP 192.168.53.99.51992 > 192.168.60.5.23: Flags [.], ack 28, win
502, options [nop,nop,TS val 3782230672 ecr 3560914663], length 0
05:07:48.292450 IP 192.168.60.5.23 > 192.168.53.99.51992: Flags [.], ack 28, win
509, options [nop,nop,TS val 3560914665 ecr 3782230671], length 0
05:07:48.292464 IP 192.168.53.99.51992 > 192.168.60.5.23: Flags [P.], seq 28:37,
ack 28, win 502, options [nop,nop,TS val 3782230674 ecr 3560914665], length 9 [telnet SB NAWs IS 0x50 0 0x18 SE [|telnet]
by 05:07:48.294330 IP 192.168.60.5.23 > 192.168.53.99.51992: Flags [P.], seq 28:46,
ack 28, win 509, options [nop,nop,TS val 3560914667 ecr 3782230672], length 18
[|telnet SB TSPEED SEND SE, SB NEW-ENVIRON SEND SE, SB TERMINAL TYPE SEND SE [|telnet]
IP, 05:07:48.294339 IP 192.168.53.99.51992 > 192.168.60.5.23: Flags [.], ack 46, win
502, options [nop,nop,TS val 3782230676 ecr 3560914667], length 0
05:07:48.296158 IP 192.168.60.5.23 > 192.168.53.99.51992: Flags [.], ack 37, win
509, options [nop,nop,TS val 3560914669 ecr 3782230674], length 0
05:07:48.296170 IP 192.168.53.99.51992 > 192.168.60.5.23: Flags [P.], seq 37:71,
ack 46, win 502, options [nop,nop,TS val 3782230678 ecr 3560914669], length 34
[|telnet SB TSPEED IS 0x33 0x38 0x34 0x30 0x30 0x2c 0x33 0x38 0x34 0x30 0x30 SE,
SB NEW-ENVIRON IS SE, SB TERMINAL TYPE IS 0x78 0x74 0x65 0x72 0x6d SE [|telnet]
) 05:07:48.300564 IP 192.168.60.5.23 > 192.168.53.99.51992: Flags [.], ack 71, win
509, options [nop,nop,TS val 3560914672 ecr 3782230678], length 0
```

The packet capture demonstrates successful VPN tunnel operation by showing a complete Telnet session between client 192.168.53.99 and private network server 192.168.60.5. The TCP three-way handshake, Telnet protocol negotiation, and bidirectional data flow confirm that the tunnel properly routes traffic between the 192.168.53.0/24 and 192.168.60.0/24 networks. 3 way handshake.

1. 192.168.53.99.51992 → 192.168.60.5.23 [SYN] (Connection initiation)
2. 192.168.60.5.23 → 192.168.53.99.51992 [SYN-ACK] (Server acknowledges)
3. 192.168.53.99.51992 → 192.168.60.5.23 [ACK] (Connection established)

## Task6: Tunnel-Breaking Experiment

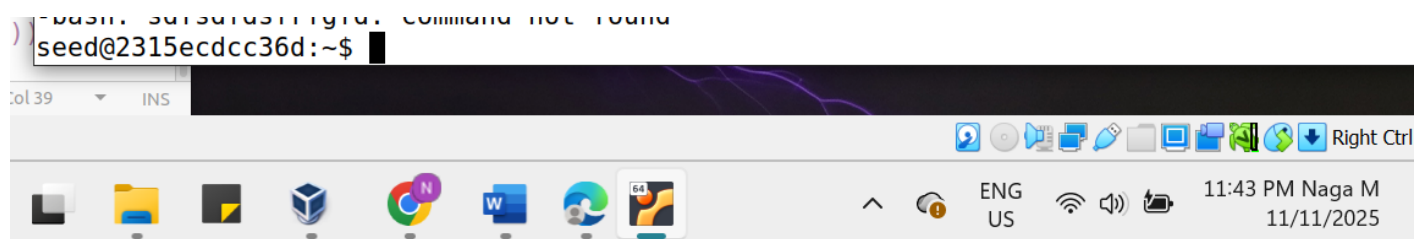
After completing the above task, On Host U, break the connection. While keeping the telnet connection alive, we break the VPN tunnel by stopping the tun client.py. (Note: I did the entire task 5 with new container Ids, and I'm just adding the last telnet output what needed for this task)

```
-4 From tun ==>: 192.168.53.99 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.53.99
^CTraceback (most recent call last):
  File "tun_client.py", line 34, in <module>
    ready, _, _ = select.select([sock, tun], [], [])
KeyboardInterrupt

root@04707045203f:/volumes# python3 tun_client.py
Interface Name: NagaM
```

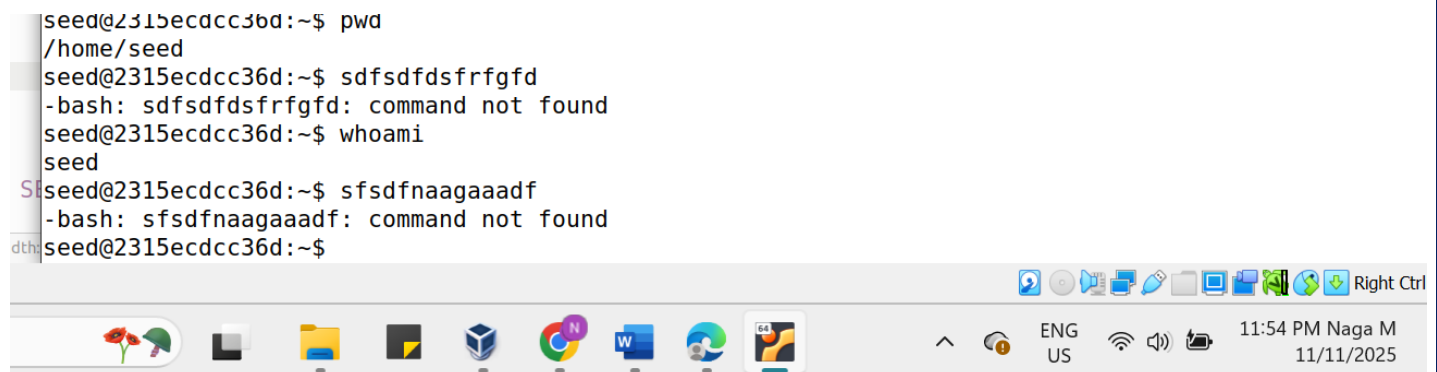
After breaking the connection, I tried typing in the telnet window but nothing was typed. I didn't see what I typed, maybe the connection was broken.

```
-bash: sudo: command not found
seed@2315ecdcc36d:~$
```



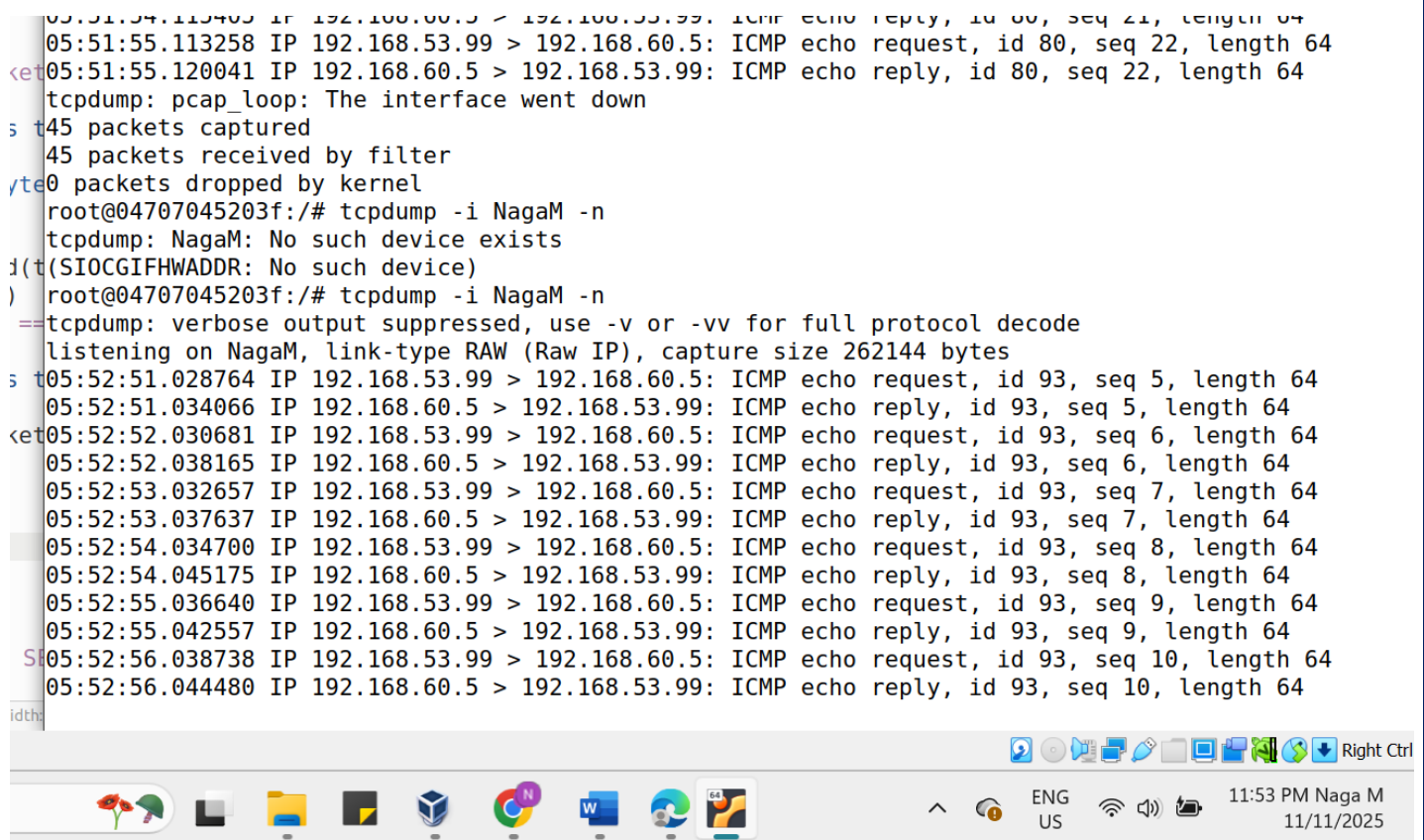
But after making the connection I'm able to see what I typed in that

```
seed@2315ecdcc36d:~$ pwd
/home/seed
seed@2315ecdcc36d:~$ sdfsdfrfgfd
-bash: sdfsdfrfgfd: command not found
seed@2315ecdcc36d:~$ whoami
seed
seed@2315ecdcc36d:~$ sfsdfnaagaaadf
-bash: sfsdfnaagaaadf: command not found
seed@2315ecdcc36d:~$
```



From the tcpdump, we can observe how much time the interface was went down. Approximately 56 seconds. This demonstrates that while the TUN interface can be unstable, it has self-recovery capabilities suitable for VPN applications.

```
05:51:54.113403 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 80, seq 21, length 64
05:51:55.113258 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 80, seq 22, length 64
05:51:55.120041 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 80, seq 22, length 64
tcpdump: pcap_loop: The interface went down
s t 45 packets captured
45 packets received by filter
yte 0 packets dropped by kernel
root@04707045203f:/# tcpdump -i NagaM -n
tcpdump: NagaM: No such device exists
d(t(SIOCGIFHWADDR: No such device)
) root@04707045203f:/# tcpdump -i NagaM -n
==tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on NagaM, link-type RAW (Raw IP), capture size 262144 bytes
s t 05:52:51.028764 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 93, seq 5, length 64
05:52:51.034066 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 93, seq 5, length 64
05:52:52.030681 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 93, seq 6, length 64
05:52:52.038165 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 93, seq 6, length 64
05:52:53.032657 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 93, seq 7, length 64
05:52:53.037637 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 93, seq 7, length 64
05:52:54.034700 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 93, seq 8, length 64
05:52:54.045175 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 93, seq 8, length 64
05:52:55.036640 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 93, seq 9, length 64
05:52:55.042557 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 93, seq 9, length 64
05:52:56.038738 IP 192.168.53.99 > 192.168.60.5: ICMP echo request, id 93, seq 10, length 64
05:52:56.044480 IP 192.168.60.5 > 192.168.53.99: ICMP echo reply, id 93, seq 10, length 64
```



## Task 7 — Routing experiment on Host V

Host V must route return packets to VPN Server so replies go back through tunnel.. for that On Host V (container):

Inspect default route: `ip route` , If needed delete default and add specific route to tunnel/gateway:

```
root@2f4e270e535f:/# ip route
default via 192.168.60.11 dev eth0
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.5
root@2f4e270e535f:/#
```

We delete this route and try to ping Host V to Host U. We can observe that a specific route is needed to make the packets flow:

```
[11/12/25]seed@VM:~/.../Labsetup$ docksh 2f
root@2f4e270e535f:/# ip route
default via 192.168.60.11 dev eth0
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.5
root@2f4e270e535f:/# ip route del default
root@2f4e270e535f:/# ip route
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.5
root@2f4e270e535f:/#
```

```
root@2f4e270e535f:/# ip route add 192.168.53.0/24 via 192.168.60.11
root@2f4e270e535f:/# ip route
192.168.53.0/24 via 192.168.60.11 dev eth0
192.168.60.0/24 dev eth0 proto kernel scope link src 192.168.60.5
root@2f4e270e535f:/#
```

After that run the `tun_client.py`, `tun_server.py`, ping Host V from Host V, observe the sever page, client page and tcpdump.





```
SORSH: Command not found
[11/12/25]seed@VM:~/.../Labsetup$ docksh 1f
root@lfee0586566e:/# tcpdump -i Naga0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on Naga0, link-type RAW (Raw IP), capture size 262144 bytes
06:58:55.431192 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 39, seq 1, length 64
06:58:55.435751 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 39, seq 1, length 64
06:58:56.433232 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 39, seq 2, length 64
06:58:56.434743 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 39, seq 2, length 64
06:58:57.435088 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 39, seq 3, length 64
06:58:57.436377 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 39, seq 3, length 64
06:58:58.442565 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 39, seq 4, length 64
06:58:58.446566 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 39, seq 4, length 64
06:58:59.443158 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 39, seq 5, length 64
06:58:59.446812 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 39, seq 5, length 64
06:59:00.445284 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 39, seq 6, length 64
06:59:00.449477 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 39, seq 6, length 64
06:59:01.451631 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 39, seq 7, length 64
06:59:01.458704 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 39, seq 7, length 64
```

On host V

```
192.168.60.5/24 dev eth0 proto kernel scope link src 192.168.60.5
root@2f4e270e535f:/# tcpdump -i eth0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
07:03:49.560204 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 40, seq 1, length 64
07:03:49.560514 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 40, seq 1, length 64
07:03:50.558926 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 40, seq 2, length 64
07:03:50.558940 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 40, seq 2, length 64
07:03:51.564521 IP 192.168.53.11 > 192.168.60.5: ICMP echo request, id 40, seq 3, length 64
07:03:51.564583 IP 192.168.60.5 > 192.168.53.11: ICMP echo reply, id 40, seq 3, length 64
07:03:54.765421 ARP, Request who-has 192.168.60.11 tell 192.168.60.5, length 28
07:03:54.765640 ARP, Request who-has 192.168.60.5 tell 192.168.60.11, length 28
07:03:54.765664 ARP, Reply 192.168.60.5 is-at 02:42:c0:a8:3c:05, length 28
07:03:54.765671 ARP, Reply 192.168.60.11 is-at 02:42:c0:a8:3c:0b, length 28
```

We can observe that the default route is via 192.168.60.11



## Task8: VPN Between Private Networks

### Docker setup

```
seed@VM: ~/.../Labsetup
Host1 uses an image, skipping
Host2 uses an image, skipping
Router uses an image, skipping
[11/12/25]seed@VM:~/.../Labsetup$ docker-compose -f docker-compose2.yml up
Creating network "net-192.168.50.0" with the default driver
Creating network "net-10.9.0.0" with the default driver
Creating network "net-192.168.60.0" with the default driver
WARNING: Found orphan containers (seed-attacker, victim-10.9.0.5, user2-10.9.0.7, user1-10.9.0.6) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Creating host-192.168.60.6 ... done
Creating host-192.168.60.5 ... done
Creating host-192.168.50.6 ... done
Creating client-10.9.0.5 ... done
Creating host-192.168.50.5 ... done
Creating server-router ... done
Attaching to host-192.168.60.6, client-10.9.0.5, host-192.168.50.5, host-192.168.50.6, server-router, host-192.168.60.5
host-192.168.60.6 | * Starting internet superserver inetd [ OK ]
host-192.168.50.6 | * Starting internet superserver inetd [ OK ]
host-192.168.60.5 | * Starting internet superserver inetd [ OK ]
host-192.168.50.5 | * Starting internet superserver inetd [ OK ]
```

```
seed@VM: ~/.../Labsetup
[11/12/25]seed@VM:~/.../Labsetup$ dockps
dfcf1c0fe807 server-router
f800042fcfa5 host-192.168.50.5
375e56d4815e client-10.9.0.5
e46d9e77f79a host-192.168.50.6
ce1c0af272e1 host-192.168.60.5
bdfe3af67871 host-192.168.60.6
[11/12/25]seed@VM:~/.../Labsetup$
```

And then connect all the containers which are used for this task.

```
[11/12/25]seed@VM:~/.../Labsetup$ docksh f8
root@f800042fcfa5:/# ping 192.168.60.5 -c 5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=62 time=4.51 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=62 time=3.33 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=62 time=3.01 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=62 time=1.79 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=62 time=7.18 ms

--- 192.168.60.5 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4010ms
rtt min/avg/max/mdev = 1.792/3.964/7.179/1.825 ms
root@f800042fcfa5:/#
```

```
root@375e56d4815e:/volumes# python3 tun_client.py
Interface Name: Naga0
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
```

```

[2]T Stopped nano tun_server.py
root@dfcf1c0fe807:/volumes# python3 tun_server.py
Interface Name: Naga0
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5

```

Lets capture the packets

```

se.y root@celc0af272e1:/# tcpdump -i eth0 -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), capture size 262144 bytes
17:59:12.720543 ARP, Request who-has 192.168.60.5 tell 192.168.60.11, length 28
17:59:12.720560 ARP, Reply 192.168.60.5 is-at 02:42:c0:a8:3c:05, length 28
e2.y 17:59:12.720571 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 29, seq 1,
length 64
17:59:12.720585 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 29, seq 1, l
length 64
17:59:13.720067 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 29, seq 2,
length 64
17:59:13.720081 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 29, seq 2, l
length 64
17:59:14.723537 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 29, seq 3,
length 64
17:59:14.723608 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 29, seq 3, l
length 64
17:59:15.722775 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 29, seq 4,
length 64
17:59:15.722790 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 29, seq 4, l
length 64
17:59:16.731387 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 29, seq 5,

```

To test the connection if done properly, we break the connection as done in the previous tasks and connect again to see that connection successfully starts again:

Stopped the client side tun program, and you can see that the ping stopped:

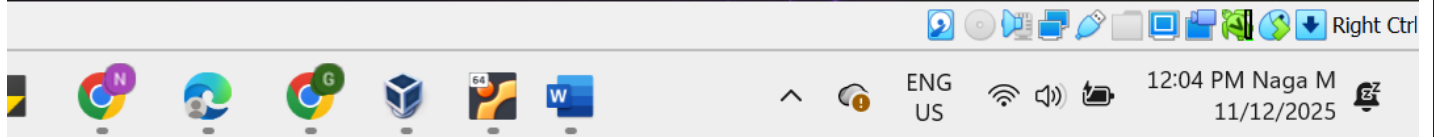
```
root@f800042fcfa5:/# ping 192.168.60.5 -c 100
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=62 time=3.49 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=62 time=4.75 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=62 time=7.24 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=62 time=4.68 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=62 time=9.46 ms
64 bytes from 192.168.60.5: icmp_seq=6 ttl=62 time=6.95 ms
```

```
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
^CTraceback (most recent call last):
  File "tun_client.py", line 41, in <module>
    ready, _, _ = select.select([sock, tun], [], [])
KeyboardInterrupt

root@375e56d4815e:/volumes#
```

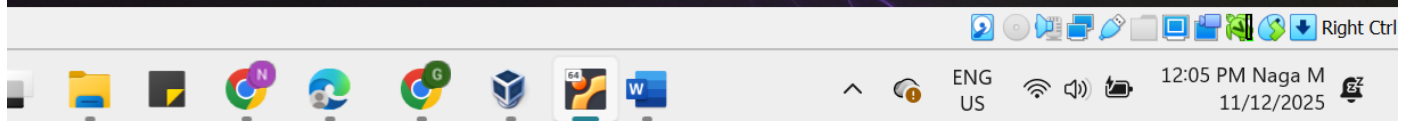


```
64 bytes from 192.168.60.5: icmp_seq=3 ttl=62 time=7.24 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=62 time=4.68 ms
64 bytes from 192.168.60.5: icmp_seq=5 ttl=62 time=9.46 ms
64 bytes from 192.168.60.5: icmp_seq=6 ttl=62 time=6.95 ms
64 bytes from 192.168.60.5: icmp_seq=46 ttl=62 time=5.28 ms
64 bytes from 192.168.60.5: icmp_seq=47 ttl=62 time=8.29 ms
64 bytes from 192.168.60.5: icmp_seq=48 ttl=62 time=8.43 ms
64 bytes from 192.168.60.5: icmp_seq=49 ttl=62 time=4.49 ms
64 bytes from 192.168.60.5: icmp_seq=50 ttl=62 time=6.31 ms
64 bytes from 192.168.60.5: icmp_seq=51 ttl=62 time=8.88 ms
```



During the connection loss, we can observe that in the tcp window

```
18:03:15.901746 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 30, seq 6, length 64
18:03:15.902682 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 30, seq 6, length 64
18:03:16.050818 ARP, Request who-has 192.168.60.11 tell 192.168.60.5, length 28
18:03:16.051232 ARP, Request who-has 192.168.60.5 tell 192.168.60.11, length 28
18:03:16.051257 ARP, Reply 192.168.60.5 is-at 02:42:c0:a8:3c:05, length 28
18:03:16.051262 ARP, Reply 192.168.60.11 is-at 02:42:c0:a8:3c:0b, length 28
18:03:56.886241 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 30, seq 46, length 64
18:03:56.886335 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 30, seq 46, length 64
18:03:57.889080 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 30, seq 47, length 64
18:03:57.889289 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 30, seq 47, length 64
18:03:58.890876 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 30, seq 48, length 64
18:03:58.891384 IP 192.168.60.5 > 192.168.50.5: ICMP echo reply, id 30, seq 48, length 64
18:03:59.890474 IP 192.168.50.5 > 192.168.60.5: ICMP echo request, id 30, seq 49, length 64
```



As our connection is ready, we telnet from Host V to Host U to see connection being established which shows our VPN was configured properly:

```
seed@VM: ~/.../Labsetup
[11/12/25]seed@VM:~/.../Labsetup$ docksh f8
root@f800042fcfa5:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
Ubuntu 20.04.1 LTS
celc0af272e1 login: seed
Password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-54-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
```

## At tcp dump

```
arpa. (43)
18:07:42.034582 ARP, Request who-has 192.168.60.11 tell 192.168.60.5, length 28
18:07:42.035594 ARP, Request who-has 192.168.60.5 tell 192.168.60.11, length 28
18:07:42.035598 ARP, Reply 192.168.60.5 is-at 02:42:c0:a8:3c:05, length 28
18:07:42.035612 ARP, Reply 192.168.60.11 is-at 02:42:c0:a8:3c:0b, length 28
18:07:46.906020 IP 192.168.60.5.23 > 192.168.50.5.43666: Flags [P.], seq 1:13, ack 25, wi
n 509, options [nop,nop,TS val 3789015382 ecr 3840681069], length 12 [telnet DO TERMINAL
TYPE, DO TSPEED, DO XDISPLOC, DO NEW-ENVIRON [|telnet]
18:07:46.910431 IP 192.168.50.5.43666 > 192.168.60.5.23: Flags [.], ack 13, win 502, opti
ons [nop,nop,TS val 3840691153 ecr 3789015382], length 0
18:07:46.910491 IP 192.168.60.5.23 > 192.168.50.5.43666: Flags [P.], seq 13:28, ack 25, w
in 509, options [nop,nop,TS val 3789015386 ecr 3840691153], length 15 [telnet WILL SUPPRE
SS GO AHEAD, DO NAWS, DO LFLOW, DONT LINEMODE, WILL STATUS [|telnet]
18:07:46.911013 IP 192.168.50.5.43666 > 192.168.60.5.23: Flags [P.], seq 25:28, ack 13, w
in 502, options [nop,nop,TS val 3840691154 ecr 3789015382], length 3 [telnet WONT XDISPLO
C [|telnet]
18:07:46.911019 IP 192.168.60.5.23 > 192.168.50.5.43666: Flags [.], ack 28, win 509, opti
ons [nop,nop,TS val 3789015387 ecr 3840691154], length 0
18:07:46.915282 IP 192.168.50.5.43666 > 192.168.60.5.23: Flags [.], ack 28, win 502, opti
ons [nop,nop,TS val 3840691158 ecr 3789015386], length 0
18:07:46.915293 IP 192.168.60.5.23 > 192.168.50.5.43666: Flags [P.], seq 28:46, ack 28, w
```

At client

```
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
From socket <==: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.50.5 --> 192.168.60.5
```

```
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
From tun ==>: 192.168.60.5 --> 192.168.50.5
From socket <==: 192.168.50.5 --> 192.168.60.5
```

## Task9: Experiment with the TAP Interface

Lets ping the any network from 192.168.53.0/24. I choose 192.168.53.50. when I tried this I didn't see any

```
seed@VM: ~/.../Labsetup
[11/12/25]seed@VM:~/.../Labsetup$ dockps
30f121dbf3b3  host-192.168.60.5
d8f9093a91a3  host-192.168.60.6
eff234fdeed5  server-router
ffb3d425f1d3  client-10.9.0.5
[11/12/25]seed@VM:~/.../Labsetup$ docksh 30
root@30f121dbf3b3:/# ping 192.168.53.50
PING 192.168.53.50 (192.168.53.50) 56(84) bytes of data.
^C
--- 192.168.53.50 ping statistics ---
14 packets transmitted, 0 received, 100% packet loss, time 13347ms
```

```
seed@VM: ~/.../Labsetup
root@b90847a251cf:/# ping 192.168.53.1
PING 192.168.53.1 (192.168.53.1) 56(84) bytes of data.
From 192.168.53.99 icmp_seq=1 Destination Host Unreachable
From 192.168.53.99 icmp_seq=2 Destination Host Unreachable
From 192.168.53.99 icmp_seq=3 Destination Host Unreachable
From 192.168.53.99 icmp_seq=4 Destination Host Unreachable
From 192.168.53.99 icmp_seq=5 Destination Host Unreachable
From 192.168.53.99 icmp_seq=6 Destination Host Unreachable
From 192.168.53.99 icmp_seq=7 Destination Host Unreachable
From 192.168.53.99 icmp_seq=8 Destination Host Unreachable
From 192.168.53.99 icmp_seq=9 Destination Host Unreachable
From 192.168.53.99 icmp_seq=10 Destination Host Unreachable
From 192.168.53.99 icmp_seq=11 Destination Host Unreachable
From 192.168.53.99 icmp_seq=12 Destination Host Unreachable
From 192.168.53.99 icmp_seq=13 Destination Host Unreachable
From 192.168.53.99 icmp_seq=14 Destination Host Unreachable
From 192.168.53.99 icmp_seq=15 Destination Host Unreachable
From 192.168.53.99 icmp_seq=16 Destination Host Unreachable
From 192.168.53.99 icmp_seq=17 Destination Host Unreachable
From 192.168.53.99 icmp_seq=18 Destination Host Unreachable
From 192.168.53.99 icmp_seq=19 Destination Host Unreachable
From 192.168.53.99 icmp_seq=20 Destination Host Unreachable
From 192.168.53.99 icmp_seq=21 Destination Host Unreachable
From 192.168.53.99 icmp_seq=22 Destination Host Unreachable
```

Leave the ping and Lets try arping

```
root@b90847a251cf:/# arping -I Naga1 192.168.53.1
ARPING 192.168.53.1
Timeout
Timeout
```



We can see the following output in client window.

```
root@b90847a251cf:/volumes# python3 tap_client.py
Interface Name: Naga1
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
Ether / ARP who has 192.168.53.1 says 192.168.53.99 / Padding
```

```
root@b90847a251cf:/# arping -I Naga1 192.168.53.33
ARPING 192.168.53.33
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=0 time=8.928 usec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=1 time=9.462 usec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=2 time=13.077 usec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=3 time=17.860 usec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=4 time=2.395 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=5 time=4.528 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=6 time=1.791 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=7 time=5.710 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=8 time=986.864 usec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=9 time=1.292 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=10 time=2.862 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=11 time=3.913 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=12 time=2.932 msec
42 bytes from aa:bb:cc:dd:ee:ff (192.168.53.33): index=13 time=5.658 msec
^Z
[2]+  Stopped                  arping -I Naga1 192.168.53.33
root@b90847a251cf:/#
```

Lets arping 1.2.3.4

```
root@b90847a251cf:/# arping -I Naga1 1.2.3.4
```

```
ARPING 1.2.3.4
```

```
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=0 time=1.074 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=1 time=3.836 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=2 time=3.518 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=3 time=2.944 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=4 time=4.873 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=5 time=5.330 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=6 time=3.522 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=7 time=1.669 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=8 time=1.032 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=9 time=1.275 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=10 time=4.928 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=11 time=1.378 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=12 time=1.032 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=13 time=2.702 msec
42 bytes from aa:bb:cc:dd:ee:ff (1.2.3.4): index=14 time=2.695 msec
```

```
^C
```

```
--- 1.2.3.4 statistics ---
```

```
15 packets transmitted, 15 packets received, 0% unanswered (0 extra)
```

```
rtt min/avg/max/std-dev = 1.032/2.787/5.330/1.466 ms
```

```
root@b90847a251cf:/#
```

Right Ctrl



ENG  
INTL



12:57 PM Naga M  
11/12/2025

```
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
```

```
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
```

```
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
```

```
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
```

```
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
```

```
-----
Ether / ARP who has 1.2.3.4 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 1.2.3.4
```

Right Ctrl



ENG  
INTL



12:57 PM Naga M  
11/12/2025

```
root@b90847a251cf:/volumes# python3 tap_client.py
Interface Name: Naga1
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
Ether / ARP who has 192.168.53.33 says 192.168.53.99 / Padding
***** Fake response: Ether / ARP is at aa:bb:cc:dd:ee:ff says 192.168.53.33
-----
```

Right Ctrl

12:55 PM Naga M  
11/12/2025

### From the above output we can observe

the ARP request detected on the TAP interface. our code then crafts a fake ARP reply and prints the "Fake response..." line. The repetition of this pattern means ARP requests are being broadcast and your TAP interface script is catching and replying to them each time.